Chapter 8. Main memory. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS08-E01. Page translation

1. 4 KiB is 2^{12} bytes. The offset is `0x2A7C & 0xFFF = 0xA7C` and the frame base is `9*0x1000 = 0x9000`, giving physical address `0x9A7C`.
2. The write fails the mapping's write-permission check. The forbidden write must not take effect before the fault is reported. A logical COW mapping could have a separate recovery protocol, which this simple entry does not specify.
3. No. Depending on the descriptor and OS model, it may represent an unpopulated legal range, a swapped-out page, or an illegal address. Consult the logical-region and backing rules; do not equate one architecture's valid or present bit with every one of these facts.

Validation criterion. Retain offset width and permission checks throughout the arithmetic.

OS08-E02. Effective access time

1. A hit costs $2 + 100 = 102$ ns. A miss costs $2 + 100 + 100 = 202$ ns, comprising one entry read and one target-data access. Thus $EMAT = 0.95 \cdot 102 + 0.05 \cdot 202 = 107$ ns. The returned frame number is not another independent memory access.
2. $EMAT = 0.80 \cdot 102 + 0.20 \cdot 202 = 122$ ns.
3. With fault probability f, use (1-f)*EMAT_no_fault+f*F if F is the complete elapsed cost of a faulting reference including its eventual completion. If F counts only extra service, add the normal completion cost separately. State the convention to prevent double-counting.

Validation criterion. Enumerate the actual hit and miss operations before weighting them.

OS08-E03. Page-size tradeoffs

1. Each object occupies ceil(33/4)=9 pages, or 36 KiB. The unused tail is exactly 3 KiB per object under the stated alignment and no-packing rule, totaling 300 KiB. Object packing would change that conclusion.
2. For the same mapped virtual range, larger pages need fewer entries and cover more bytes per TLB entry. The actual number of table levels depends on the architecture.
3. A 64 KiB allocation wastes 64−33=31 KiB per object, or 3100 KiB across 100 objects. Smaller pages need not minimize total memory: page-table storage also counts. Translation misses and fault-service counts or time are separate execution costs, not bytes to add to the memory footprint; their number need not always increase.

Validation criterion. Include both space and translation costs, with their allocation assumptions.

OS08-E04 An unfamiliar x86-64 walk

1. Bits63:48 are 0 and bit 47=0, so the address is canonical. Indices are 0x024/0x0D1/0x0B3/0x189 and offset 0xABC.
2. Index×8 gives offsets 0x120/0x688/0x598/0xC48. Entry addresses are 0x7120/0x8688/0x9598/0xAC48. Plausible entries 0x8007/0x9007/0xA007/0x12AB007 encode the specified next bases and present/RW/user flags.
3. PA=0x12AB000+0xABC=0x12ABABC.
4. PTE 0x12AB005 preserves present/user but clears RW. After the specified TLB invalidation, the CPL=3 read still reaches 0x12ABABC; a user write protection-faults before modifying data. Every upper level must also allow the operation. Supervisor writes depend on CR0.WP and other conditions, so this user conclusion cannot simply be generalized.

OS08-E05 Canonical, present, and permissions

1. A has bits63:48=0 and bit 47=0, so is canonical. Indices 0x048/0x1A2/0x167/0x001, offset 0x234.
2. D has bits63:48=0 but bit 47=1. It rejects before any table access. B stops at the absent PML4E without reading a lower table.
3. A uses PML4 index 0x048 rather than E04's 0x024. Its entries are unspecified, so read, write and fetch results cannot be determined. B page-faults on all three at the absent PML4E. Under NXE=1 and the specified effective user/RW rights, C permits data read/write but fetch incurs a page protection fault. D rejects all three before walking.
4. Present only establishes translation availability. RW controls writes and NX distinguishes fetch from data. Record canonical, present, U/S, RW, NX and operation result, stopping at the first failing stage. When NXE=0, bit 63 of a present entry is reserved; setting it causes a reserved-bit page fault rather than an ordinary NX execution-protection result. The C result assumes NX support and NXE=1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS08-E06 ARM layout, domain, and AP

1. N=0 roots have 4096×4=16 KiB storage and 16 KiB alignment. P1 spans 0x14000–0x17FFF,P2 0x18000–0x1BFFF; they are aligned/disjoint. L2 tables need 256×4=1 KiB and 1 KiB alignment,with disjoint ranges 0x1C000–0x1C3FF and0x1C400–0x1C7FF. P1 indices(0x4,0xA6) give entries 0x14010/0x1C298. P2 indices(0x7,0xC8) give 0x1801C/0x1C720.
2. L1 type01 means coarse,bases 0x1C000/0x1C400,domain bits 8:5=3. L2 frame 0x54000; AP2 bit 9=0,AP1 bit 5=1,AP0 bit 4=0,so AP010. XN=0,nG=1,TEX=001,C=B=0. Client/AFE0 grants PL0 read and denies PL0 write for both mappings.
3. Invalidate only P1's L2 entry and complete required TLB invalidation, barriers and synchronization on affected cores. P1 read translation-faults; P2's independent mapping still allows read. With exactly the two mappings specified, mapping count goes 2→1 and the frame cannot yet be reclaimed. After P2 is also unmapped and its TLB maintenance completes, reclaim only once all other lifetime references are absent. A mapping count is not a general frame refcount.
4. Manager 11 bypasses AP,so the same descriptor no longer proves write fault. AFE1 makes AP0 an access flag and uses AP[2:1] for rights. The unchanged word has flag0 and can access-flag fault under software management. Recompute rather than reuse AFE0's result. Wrong word 0x54802 decodes AP000 and faults even on PL0 read. Word0x54022 has AP010 but Strongly-ordered TEX/C/B=0/0/0; XN0 alone does not prove executable access. Storage,raw fields,domain,AFE,and TLB state all belong to the result.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study